



Tecnologico de Monterrey
Escuela de Ingeniería y Ciencias

TC2008B

Modelación de Sistemas Multiagentes con Gráficas Computacionales

Final Integrative Project

**Distributed Coordination of AGVs in a Digital Twin of a
Distribution Centre**

*BFS routing and independent Q Learning for battery aware mission assignment in a
fleet of four AGVs*

Team .gitignore

Student name	Student ID
Luis Enrique Castillo Carrillo	A00842390
Ricardo Gomez Novelo	A00843383
Duziel Monarrez Garcia	A01743905
Brian Roberto Gómez Martínez	A00841404
Fabrizio Emanuel Mariano Arango	A00842796
Carlos Emiliano Nava Flores	A00842650

Case study provided by the e80 Group
11 September 2026

1. Introduction

1.1 Logistics automation and distribution centres

A modern distribution centre is a continuous flow system. Pallets leave production lines, are buffered in storage racks and are loaded onto trucks at the docks, and the internal transport connecting those zones is increasingly performed by Automated Guided Vehicles. Automation does not remove the coordination problem, it relocates it: a fleet sharing one floor has to decide, many times per minute and with no human in the loop, which vehicle takes which order, which route it follows, and when it stops working to recharge. The three decisions are coupled, because accepting a distant order consumes charge needed later, occupying one of a few chargers denies it to everyone else, and a route that is optimal in an empty aisle stops being optimal once another vehicle, a pedestrian or a closure occupies it.

This is why such a fleet is naturally modelled as a Multi Agent System. Each AGV is autonomous, with a local view of the plant, its own battery and its own commitments, and the behaviour of the plant is not written anywhere: it emerges from the interaction of those agents through a shared environment and a negotiation protocol. The engineering question is not how to program the fleet, but what decision rule to give each vehicle so that the aggregate behaviour is good.

1.2 Problem statement

The starting point is the fleet built for module M3, which coordinates correctly but decides badly. Its AGVs are already decentralised: they perceive, they bid in an auction and they resolve their own physical conflicts. What they lack is any notion of the future consequence of a decision. Three failures were measured on that baseline over 20 shifts of 400 ticks, before anything was proposed.

A. AGVs strand themselves. The rule inspects the battery only while an AGV is Available, against a fixed threshold of 45 per cent, so a vehicle that accepts a mission at 46 per cent and must then drive to the pickup, carry the load and reach a charger does not arrive. Measured: 3.45 run flat events per shift, each immobilising a vehicle for 25 ticks and pushing its mission back into the queue.

B. Charging is all or nothing and badly timed. An AGV charges only after dropping below the threshold, and then always to 100 per cent, occupying one of three stations for a long block. It never tops up while the queue is empty and never leaves early when a partial charge would suffice.

C. Mission selection never asks whether the mission is reachable. The bid combines distance, battery and workload through three weights chosen by hand, and nothing in it answers the question that matters: after this mission, can I still reach a charger?

The three are one failure. In every case the cost of a decision is paid much later, and the agent cannot compute that cost in closed form, because it depends on what three other independently deciding agents, a set of pedestrians and a stream of aisle closures do in the meantime. That is a sequential decision problem with unknown dynamics.

1.3 Objectives

General objective. Determine whether replacing the reactive decision layer of the baseline fleet with a learned, state aware policy improves the performance of the Multi Agent System, and quantify that improvement under conditions strict enough for the answer to be attributable to the decision layer and to nothing else.

Specific objectives. (1) Measure the failures of the baseline before proposing anything. (2) Design a decision layer that represents the delayed consequence of a decision explicitly, through an energy reserve computed from real route lengths. (3) Keep environment, agents, auction, routing and battery physics identical between the two strategies, and verify that equivalence numerically. (4) Evaluate both strategies on 20 paired scenarios never seen during training, reporting per seed win counts alongside the means. (5) Report metrics that measure the decentralisation itself, not only fleet aggregates a central controller could also produce. (6) Report the metrics that worsened alongside those that improved, and state the conditions under which the conclusion holds.

1.4 Case study

The scenario is the e80 Group logistics case. A distribution centre with three production lines, six storage racks in three double columns, three truck docks, three charging stations, structural columns and two restricted zones. Of the 1440 cells, 865 (60.1 per cent) are traversable aisle, and the plant holds 27 elements and 75 pallet positions.

A shift lasts 400 ticks. Four AGVs serve it against three chargers, so charging is a genuinely contested resource. About 27 orders are published per shift, most after the shift has started, each moving a pallet between a production line, a rack and a dock. Three pedestrians walk the aisles and aisles close for 15 to 35 ticks, so the floor an AGV planned across is not the floor it drives across. Low battery emerges from the movement the AGVs choose to perform.

2. Methodology

2.1 Multi agent modelling

The system is implemented in Python with the AgentPy framework. It contains one class of autonomous agent, the AGV, and three structures the agents act upon: the environment, the pallets and the Mission Manager. Within a tick the order of operations is fixed: publish new missions, advance the dynamic events, let every agent at a decision point decide and negotiate, arbitrate movement, service arrivals and charging, and update every battery. The essential choice is that no component holds the fleet policy. The Mission Manager publishes missions and reports who won each auction but never selects an executor, and each AGV decides for itself from local information only. The behaviour of the plant is therefore emergent, which is what makes this a Multi Agent System rather than a scheduler with four actuators.

2.2 AGV agents, pallets and missions

Each AGV holds the state $s_i = (\text{position}, \text{battery}, \text{status}, \text{mission})$, where status is one of Available, ToPickup, Transporting, Charging or Stranded. Status determines whether

the agent is at a decision point at all, a vehicle carrying a pallet is committed and does not bid, which is itself a coordination mechanism, since it cannot compete for work it could not perform. Its perception is strictly local: own position and battery, the broadcast list of pending missions, which chargers are occupied, and the pedestrians and closed cells inside the traversable mask. It never reads the state of another AGV.

Pallets are points inside zones rather than solid blocks, because an AGV stops on the aisle cell adjacent to a load, and each pallet advances through stored, reserved, being transported and delivered, plus waiting for pickup if its carrier ran flat. A mission is the tuple $M_k = (O_k, D_k, P_k, S_k)$ with origin pallet, destination pallet, priority and status, advancing Pending, then Assigned, then Completed, and returning to Pending if the carrier runs flat. Missions follow the three flows of the case study: production line to rack or dock, rack to dock, and dock to rack.

2.3 Mission manager

The Mission Manager replays an arrival schedule generated before the simulation starts. Each tick publishes the missions whose arrival time has come, records acceptance, completion and release, and keeps the pending pool. Its role is deliberately limited: it is a bulletin board and a registry, not an allocator. It never chooses an executor and computes no assignment, and when an auction closes it reports the argmax of the bids received.

2.4 Communication, collaboration and negotiation

Communication. Once per tick the Mission Manager broadcasts every pending mission, as $Message_k = (MissionID, O_k, D_k, P_k)$, to every AGV at a decision point, and each replies with a bid. Because the bid is a function of the agent's own battery and workload, the auction is also how AGVs tell each other how loaded they are, so no separate status protocol is needed. Positions are communicated implicitly, through the shared occupancy the movement arbiter and the BFS replanner read.

Negotiation. Missions are resolved in publication order and the bid is the utility

$$U(i,k) = -w_d \cdot D(i,k) + w_b \cdot B_i - w_l \cdot L_i, \quad w_d = 2.0, \quad w_b = 0.5, \quad w_l = 20.0, \\ AGV^* = \text{argmax over } i \text{ of } U(i,k)$$

where $D(i,k)$ is the BFS route length from AGV i to the origin of mission k , B_i its battery and L_i its accumulated workload, and the winner is the highest bidder among those not already committed in the same tick. The weights and the mechanism are **identical in both strategies compared in this report**, which makes the experiment a test of the decision layer and not of the auction.

Collaboration. A mission leaves the pool the instant it is accepted, so no two AGVs execute the same one. A busy or charging AGV is not at a decision point and never competes for work it cannot perform. A mission whose carrier runs flat returns to the pool for someone else; and the movement arbiter makes AGVs yield to each other.

2.5 Environment

The plant is described by two structures generated from one parameter dictionary: a list of elements (id, kind, x, y, w, h) covering racks, production lines, docks, chargers, walls

and obstacles, and an occupancy grid from which the traversable mask is obtained, so a different layout comes from changing numbers rather than code. The environment is dynamic in three ways, all generated from the scenario seed before the run starts: mission availability changes over time, three pedestrians walk the aisles and may not be entered or stepped into, and aisles close for 15 to 35 ticks, invalidating routes already planned across them.

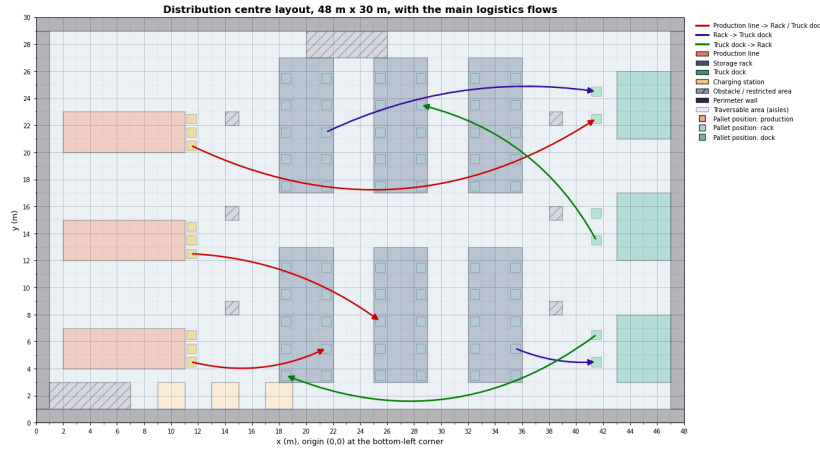


Figure 1. Layout of the distribution centre on the 1 m occupancy grid with the three logistics flows of the case study. Production lines on the left, six storage racks in the centre, three truck docks on the right, three charging stations at the bottom.

2.6 Routing, movement and battery

Routing and deciding are different questions answered by different algorithms: BFS answers how the AGV should reach the destination, and Q Learning answers what it should decide to do. BFS was chosen over A* and Dijkstra on the merits of this floor, a four connected grid with a uniform edge cost of 1 m per cell and only 865 traversable cells. Under uniform cost the priority queue of Dijkstra degenerates into a FIFO queue, which is BFS with extra overhead, and A* would only add a heuristic to a search space small enough that a full expansion takes about 3 ms. A distance oracle caches the distance field from each source, so repeated queries are free.

Task allocation is decentralised, but movement cannot be, because two independently computed shortest paths can still cross. A movement arbiter resolves this every tick: every AGV proposes its next cell, head on swaps are cancelled on both sides, same cell claims go to whoever is already parked there, and only then does anyone move. A vehicle blocked for two consecutive ticks replans.

Battery falls by $c_d = 0.6$ per cell moved and by $c_{idle} = 0.15$ per tick spent available, and rises by $R_c = 10.0$ per tick while parked at a charger, capped at 100. An idle AGV is therefore not free to park, which makes standing still a decision with a real cost. At zero charge the vehicle is immobilised for 25 ticks, its mission returns to the pool and it comes back into service at 30 per cent: that is the run flat event which dominates the diagnosis of the baseline.

2.7 Intelligent decision making technique

The decision has three properties that determine the technique. Its cost is paid much later, so it is sequential; the agent observes the variables that matter, so the state is available; and the transition probabilities are unknown, because they depend on three other independently deciding agents, on stochastic pedestrians and on aisle closures. That is a Markov Decision Process whose model cannot be written down, so Value Iteration is unavailable and model free reinforcement learning is required. Q Learning is its simplest form:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

State. The local observation is discretised into 288 states, $s_i = (b_i, d_i, n, p_i, k_i)$:

Symbol	Meaning	Buckets
b_i	Own battery	below 25, 25 to 50, 50 to 75, 75 and above (4)
d_i	BFS route length to the closest pending mission	12 or less, 28 or less, more than 28, none pending (4)
n	Pending mission backlog	0, 1 to 2, 3 or more (3)
p_i	Energy reserve after the full round trip	negative, 0 to 15, above 15 (3)
k_i	Parked on a charger	no, yes (2)

The reserve p_i is where routing and decision meet, and it is the single most important modelling decision of the project. It is the charge that would be left **after** the complete journey, from three cached BFS distances:

$$p_i = B_i - c_d \cdot [D(p_i, O_k) + D(O_k, D_k) + \min_{c} D(D_k, c)]$$

A negative reserve means that taking this mission strands the AGV. A hand written rule could encode that threshold, but not how to trade it off against the backlog, the distance to the mission and the contention for three chargers, and that trade off is what has to be learned.

Actions and reward. The action set is { BID_NEAREST, BID_PRIORITY, WAIT, GO_CHARGE }. BID_NEAREST bids on the pending mission with the shortest BFS route and BID_PRIORITY on the highest priority one. GO_CHARGE routes to a charging station, WAIT stays available, and at a charger it means keep charging, so choosing to bid instead is the decision to leave early with a partial charge. WAIT is deliberately illegal for an available AGV while work is pending, because an unconditional WAIT lets a policy dodge every risk by never committing. An action lasts a variable number of ticks, so the problem is a semi Markov Decision Process: a decision taken at t_0 is evaluated only when the agent returns to a decision point at t_1 , charged with everything that happened in between.

$$r = R_{done} \cdot \Delta_{completed} - c_t \cdot \Delta t - c_d \cdot \Delta_{cells} - P_{strand} \cdot \Delta_{flat} - c_{idle} \cdot \Delta t \cdot 1[a = WAIT \text{ and } n > 0]$$

$$Q_i(s_0, a_0) \leftarrow Q_i(s_0, a_0) + \alpha [r + \gamma^{\Delta t} \cdot \max_{\text{legal } a'} Q_i(s_1, a') - Q_i(s_0, a_0)]$$

$$R_{done} = 100, \quad c_t = 0.5, \quad c_d = 0.3, \quad P_{strand} = 150, \quad c_{idle} = 4, \quad \alpha = 0.10, \quad \gamma = 0.999, \quad \epsilon \text{ from } 1.0 \text{ to } 0.05$$

The discount is load bearing and is not the usual 0.99: at $\gamma = 0.99$ the continuation value of a 60 tick mission is multiplied by about 0.55 while a one tick WAIT keeps 0.99 of it, so discounting rewards not committing and the policy idles, whereas at $\gamma = 0.999$ the factor is about 0.94.

Independent learning. Each AGV owns a separate Q table updated only from its own experience: no shared parameters, no shared memory, no central critic, with agents influencing each other only through the environment and the auction. The cost is nonstationarity, since while AGV 1 learns, the behaviour of AGV 2 keeps changing underneath it. A single centralised policy was considered and rejected before implementation: with the same encoding it would need 288 to the power 4 joint states and 256 joint actions, about 1.76 million million table entries against 4608 for four independent tables, and one controller ingesting the full fleet state every tick is also a communication bottleneck and a single point of failure. It is worth being explicit, because the two axes are easy to confuse: both strategies compared in this report are decentralised. The baseline is a decentralised rule based fleet, not a central controller, and centralised control is a third design that was never implemented.

3. Results

3.1 Experimental design

Both strategies were evaluated on the same 20 scenarios of 400 ticks with 4 AGVs and 3 chargers. Equivalence was obtained by construction and then verified. By construction the whole world timeline, that is initial positions and batteries, order stream, pedestrian trajectories and aisle closures, is generated from the scenario seed before the run starts, so it cannot depend on what the AGVs do. By verification both strategies were re run on 10 seeds and compared field by field: order streams, pedestrian positions and closed cells at every tick and initial batteries were identical in 10 of 10 seeds. Training used seeds 10000 to 10999 and evaluation seeds 0 to 19, so the sets are disjoint and the results measure generalisation rather than memorisation. Training ran 1000 episodes in 56 seconds, after which each AGV had visited between 103 and 108 of the 288 possible states, with throughput rising from about 2.0 to 4.75 missions per 100 ticks and run flat events falling from 12.6 to 2.7 per episode (Figure E1). Because the scenarios are paired, a per seed win count is reported next to every mean.

3.2 Computational results

$$\text{Throughput} = N_{\text{completed}} / T_{\text{simulation}} \quad \text{Mean mission time} = (1 / N) \cdot \sum (T_k^{\text{complete}} - T_k^{\text{start}})$$

$$\text{Utilization}_i = (T_i^{\text{active}} / T_{\text{simulation}}) \times 100 \text{ per cent} \quad \text{Improvement} = (P_{\text{proposed}} - P_{\text{baseline}}) / P_{\text{baseline}} \times 100$$

Mission time is reported twice because the two versions answer different questions. Cycle time runs from publication of the order to delivery and therefore includes queueing, which is what a customer experiences; execution time runs from acceptance to delivery. For metrics where lower is better a negative change is an improvement, so Table 1 states the direction of every metric explicitly.

Metric	Better when	Baseline	Proposed	Change	Seeds won	Verdict
Missions completed	higher	16.15 $\pm$ 3.32	20.70 $\pm$ 3.02	+28.2 %	15/20	better
Throughput (missions / 100 ticks)	higher	4.04 $\pm$ 0.83	5.17 $\pm$ 0.75	+28.2 %	15/20	better
Mean cycle time (ticks)	lower	142.75 $\pm$ 34.42	95.80 $\pm$ 22.10	-32.9 %	19/20	better
Mean execution time (ticks)	lower	56.92 $\pm$ 8.63	46.15 $\pm$ 4.60	-18.9 %	17/20	better
AGV utilization (per cent)	higher	69.05 $\pm$ 4.15	66.22 $\pm$ 3.18	-4.1 %	6/20	worse
Distance per mission (cells)	lower	85.43 $\pm$ 15.03	66.95 $\pm$ 7.13	-21.6 %	18/20	better
Energy per mission (per cent)	lower	51.60 $\pm$ 9.06	40.52 $\pm$ 4.31	-21.5 %	18/20	better
Run flat events	lower	3.45 $\pm$ 1.53	1.35 $\pm$ 1.11	-60.9 %	16/20	better
Time spent charging (per cent)	information	23.52 $\pm$ 2.44	28.87 $\pm$ 2.58	+22.8 %	n/a	n/a
Idle time (per cent)	lower	2.36 $\pm$ 0.72	2.95 $\pm$ 1.02	+24.9 %	4/20	worse
Missions pending at tick 400	lower	8.15 $\pm$ 3.44	4.05 $\pm$ 2.35	-50.3 %	15/20	better
BFS replanning events	lower	63.40 $\pm$ 84.55	75.30 $\pm$ 82.47	+18.8 %	9/20	worse

Table 1. Baseline against the proposed strategy over 20 paired scenarios of 400 ticks. Means and standard deviations, on evaluation seeds unseen during training.

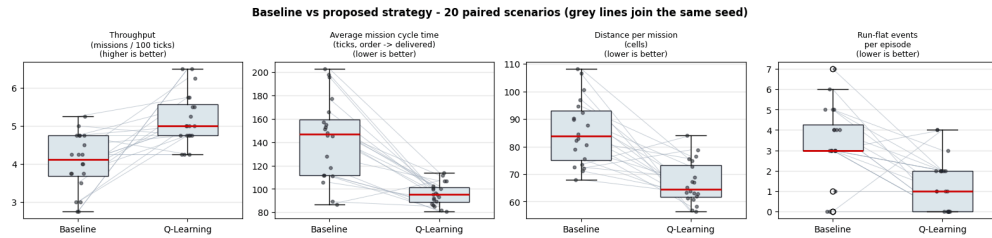


Figure 2. Paired comparison on the four decisive metrics. Each grey line joins the same seed under the two strategies, so a line sloping in the favourable direction is a seed the proposal won. The separation is clearest on cycle time, won in 19 of 20 seeds.

3.3 Metrics of the decentralised strategy

The three mandatory metrics are fleet aggregates: a central controller producing the same totals would be indistinguishable from this fleet in them. Because the proposed strategy is decentralised, a further family is reported, answering three questions only a

decentralised design has to answer. Does work distribute evenly when nothing balances it? What does coordinating through an auction cost in messages? And did the four independent learners actually produce four different policies?

Metric	Better when	Baseline	Proposed	Change	Seeds won
Workload dispersion CV across AGVs	lower	0.245 ± 0.17	0.213 ± 0.10	-12.7 %	9/20
Workload Gini across AGVs	lower	0.127 ± 0.10	0.109 ± 0.06	-13.8 %	9/20
Share carried by busiest AGV (per cent)	lower	32.96 ± 6.74	32.21 ± 4.23	-2.3 %	7/20
Share carried by quietest AGV (per cent)	higher	17.69 ± 5.10	18.75 ± 3.45	+6.0 %	10/20
Bids emitted per tick	lower	0.349 ± 0.11	0.069 ± 0.01	-80.1 %	20/20
Bids per accepted mission	lower	7.54 ± 3.19	1.18 ± 0.10	-84.3 %	20/20
Contested share of auctions (per cent)	lower	18.15 ± 4.14	9.49 ± 5.74	-47.7 %	19/20
Missions released back to the pool	lower	0.45 ± 0.67	0.40 ± 0.49	-11.1 %	6/20
BFS replans per AGV	lower	15.85 ± 21.14	18.82 ± 20.62	+18.8 %	9/20

Table 2. Metrics of the decentralised strategy over the same 20 paired scenarios.

Per agent results confirm that the fleet level gain is not one strong vehicle carrying three weak ones: missions completed rose for all four AGVs and run flat events fell for all four (Appendix E.3). Of the 98 states visited by all four agents, they select a different greedy action in 37, that is 37.8 per cent, with pairwise agreement between 71.0 and 80.6 per cent. Under a single shared table that figure would be 100 per cent by construction, so it directly measures what independent learning produced: four agents that specialised against each other rather than converging to one common rule.

3.4 Visual results and their integration with the computational results

The computational model is coupled to a graphical representation of the same state, so every number in Table 1 can be observed as behaviour on the floor. The visualisation renders the parametric layout and the pallet positions with their status colours, and on top of it the live state: each AGV as a square coloured by status with its identifier and battery, the pedestrians as triangles, and the cells closed by an event shaded in red. An animation of a full shift is produced from the recorded history and is included in the notebook.

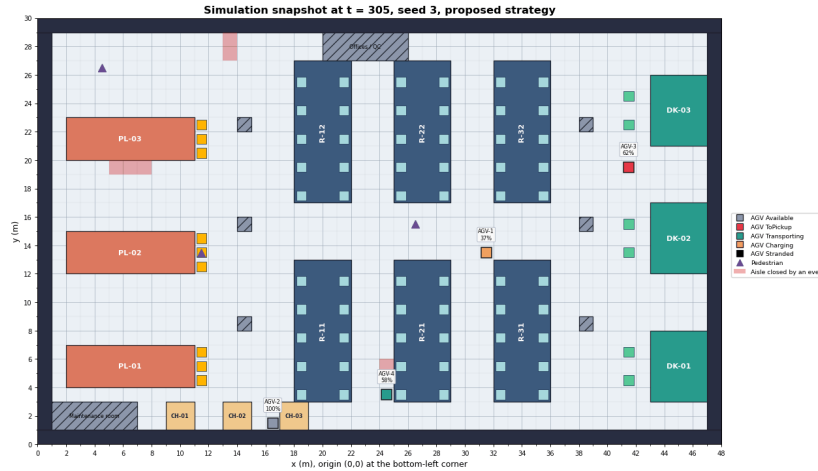


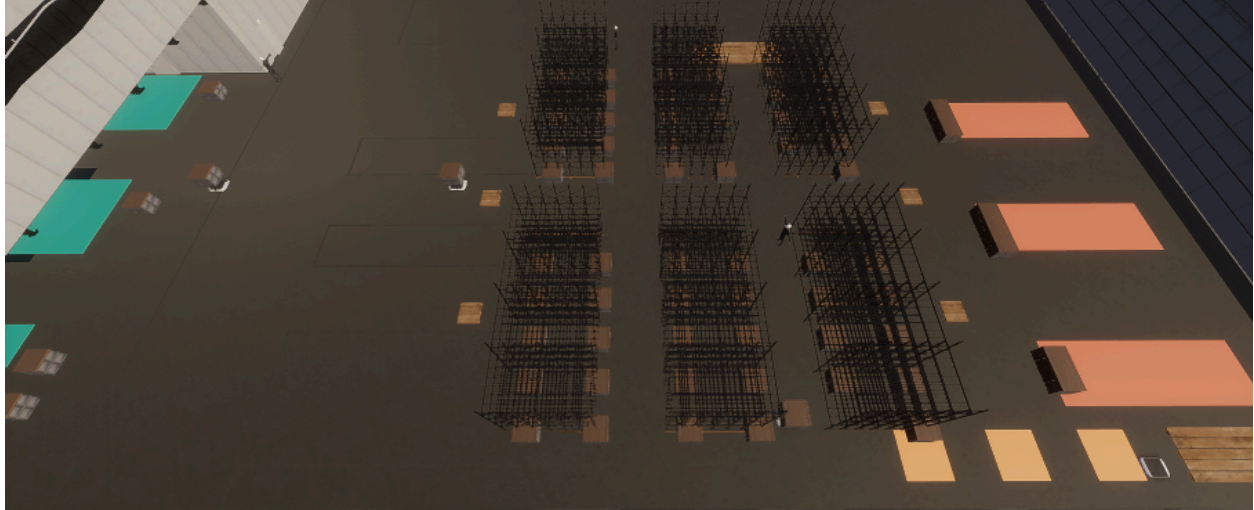
Figure 3. Snapshot at tick 305, seed 3, proposed strategy. AGV 1 is charging at 37 per cent, AGV 2 is parked and available at 100 per cent, AGV 3 is driving to a pickup at 62 per cent and AGV 4 is transporting a pallet at 58 per cent. Two pedestrians cross aisles and six cells are closed by an active event, visible as the red patches.

The two views are the same state rendered twice, and each makes the other interpretable. Table 1 reports that the proposal charges more and is less utilised yet delivers more, and Figure 3 shows why that is coherent, since at tick 305 one AGV is parked on a charger at 37 per cent instead of accepting a mission it could not finish, which counts against utilisation in the table and against nothing in the plant. Watching one shift, a run flat event looks like an anomaly, while Table 1 shows it happens 3.45 times per shift in the baseline and 1.35 under the proposal. Both views come from the same recorded history, so no discrepancy between them is possible, and the visualisation was also a verification tool: two development failures were seen as a fleet that refuses to move before they were found by measuring.

3.5 Unity digital twin

In addition to the analytical visualisation described above, the environment was also rendered as a three dimensional digital twin in Unity, built on the Unity Warehouse asset package from Unity Technologies Japan and scaled to the real dimensions of the case study. The four AGVs are represented with the Palletrobot prefab, the pedestrians with the Worker prefab, the racks with repeated Shelf prefabs, and the pallet positions with the Pallet, PlasticBox, Cardboard and Sunoko prefabs; the production line, the truck docks and the charging stations were assembled from Pedestal, Trolley and floor markers, since the asset package has no dedicated prefabs for them.

Python remains the only component that decides anything: the Unity scene only receives and displays state. The Python server opens a TCP socket and, on every simulation tick, sends a JSON message with the position, status and battery of each AGV and the position of each pedestrian. On the Unity side, the AGVUnityBridge.cs component connects to that socket, instantiates a Palletrobot the first time each AGV appears and a Worker for each pedestrian, and interpolates their movement between messages. Python must be listening before Unity connects, since Unity attempts the connection as soon as the scene starts playing.



The current version of the bridge does not yet render the AGV identifier, status, battery level or assigned mission on top of the 3D model; that information is used only to move the AGV. Displaying it, for example with a floating label or by colouring the robot according to the STATUS_COLOR palette already used in the Python figures, is the natural next step on top of this integration.

4. Discussion and analysis

4.1 The two scenarios

Scenario 1, the baseline, is carried over unchanged from M3. Every AGV at a decision point applies the same hand written rule, in this order: if parked at a charger, keep charging until 100 per cent; otherwise, if battery is 45 per cent or less, go to a charger; otherwise, if work is pending, bid on every pending mission; otherwise, wait. The rule is memoryless: it reacts to the battery at the instant it is read and never asks what the battery will be once the mission is over. Scenario 2, the proposal, replaces those four steps with $a^* = \text{argmax over legal } a \text{ of } Q_i(s_i, a)$, where the state carries the energy reserve ρ_i .

	Scenario 1: baseline	Scenario 2: proposed
Decision rule	fixed cascade of conditionals, written by hand	argmax of $Q(s,a)$, learned from experience
Information used	current battery only	battery, BFS distance to nearest mission, backlog, energy reserve after the round trip, parked or not
Horizon	the current tick	discounted future, $\gamma = 0.999$ per tick, semi Markov
Missions bid on	all pending missions at once	exactly one, the nearest or the highest priority
When it charges	only after crossing the 45 per cent threshold	whenever the learned value of GO_CHARGE is highest, including opportunistic top ups

When it leaves the charger	only at 100 per cent	as soon as the reserve is comfortable, partial charge allowed
Can it decline an unreachable mission	no, reachability is never checked	yes, a negative reserve pushes the policy to GO_CHARGE
What is tuned, and how	three bid weights, chosen by hand	4 × 288 state action values, estimated from 1000 shifts
Memory per AGV	none	one Q table, about 107 visited states of 288

Table 3. The two scenarios compared line by line.

Everything else is deliberately identical: layout and grid, scenario generation, the Mission Manager, the bid utility and its three weights, the auction, BFS routing and replanning, the movement arbiter, the battery physics, the 25 tick tow, the horizon and the fleet size. Only the decision layer changes, which is why every difference in Table 1 is attributable to that layer and nothing else.

4.2 What improved, and why

The gain came from the failure measured first. Run flat events fell from 3.45 to 1.35 per shift, a reduction of 60.9 per cent won in 16 of 20 seeds, and the learned policy shows the mechanism: its most visited rule is “battery between 25 and 50 per cent, work pending, energy reserve negative, choose GO_CHARGE”. That is exactly the decision the fixed 45 per cent threshold could not express, because the threshold reads the battery now while the reserve reads it after the round trip. Every downstream metric follows: each avoided run flat saves a 25 tick tow and prevents a mission returning to the queue, which is why missions pending at the end of the shift fall by half and why cycle time falls 32.9 per cent in 19 of 20 seeds. Cycle time improves more than execution time, 32.9 against 18.9 per cent, which locates the gain: most of it is queueing that never happens, not vehicles driving faster.

4.3 What worsened, and why it is not a contradiction

Two metrics moved against the proposal, and the most instructive observation of the project is the one that looks like a defeat. The learned fleet is less utilised, 69.05 falling to 66.22 per cent and winning only 6 of 20 seeds, and charges more, yet delivers 28.2 per cent more missions with 21.6 per cent less distance per mission. Utilisation counts the ticks in which an AGV is moving under a mission: it counts a doomed trip toward a pallet the vehicle will never reach exactly like a delivered one, and counts a 25 tick tow as neither, so the baseline earns part of its utilisation driving toward missions it cannot finish. The conclusion is methodological: in an energy constrained fleet, utilisation is a diagnostic metric and not an objective, which is why throughput was declared the primary metric before the experiment was run.

Replanning effort rose 18.8 per cent and won only 9 of 20 seeds, which makes it the weakest result in the table. The learned policy keeps AGVs in circulation more of the time, so they meet each other, the pedestrians and the closed aisles more often, which on a real fleet is compute and communication load this simulation does not charge for. It also carries the largest spread of any metric here, because it is driven by whether a closure lands on the routes in use, a property of the seed rather than of the strategy.

4.4 Reading the decentralised metrics

Work distributes evenly even though nothing balances it. No component assigns a quota, so the fleet could have collapsed onto one dominant vehicle. It does not: the busiest AGV carries 32.2 per cent of the missions and the quietest 18.8, against 33.0 and 17.7 in the baseline. The honest reading is that this is not the main effect, since only 9 of 20 seeds improve, so the fair claim is that the learned policy does not damage a balance the auction was already producing.

The decentralised layer became substantially cheaper. The baseline bids on every pending mission and emits 7.54 bids per mission actually accepted; the learned policy commits to one mission at a time and emits 1.18, a reduction of 84.3 per cent holding in 20 of 20 seeds, while contested auctions fall from 18.2 to 9.5 per cent in 19 of 20. Fewer vehicles chase the same mission, which is what a bidding mechanism is supposed to achieve and what the bid everything rule prevented. And the four policies genuinely diverged, choosing a different greedy action in 37.8 per cent of the states all four visited: each learned against the other three, and their different starting positions and charger proximities pushed them to specialise. That same mechanism is the source of the nonstationarity visible in the convergence curve.

4.5 Conditions under which the strategy performs well and badly

The advantage is an advantage under scarcity. With four AGVs against three chargers and orders arriving faster than the fleet can clear them, charging is a genuinely contested bottleneck, and managing it better is worth 28 per cent of throughput. The effect is consistent rather than lucky: cycle time improved in 19 of 20 paired scenarios, distance per mission in 18 of 20, and three independent training runs reproduced it. Three conditions delimit the claim from the other side. Replanning effort and idle share both worsened, and neither was a target. The technique proved sensitive to two choices unrelated to logistics: at $\gamma = 0.99$ the discount punished long actions enough that the fleet learned to stand still, and with an unconditional WAIT the policy converged to 65 per cent idle time and below baseline throughput. And the result rests on one trained policy, so confidence intervals over several training seeds would be needed before quoting these figures as engineering tolerances.

5. Conclusions and future work

5.1 Conclusions

The question was whether the proposed intelligent strategy improves the performance of the Multi Agent System. **The answer is yes on every metric the proposal set out to change and no on two secondary ones, and both answers come from the same table.** Over 20 paired shifts on seeds never seen during training, throughput rose 28.2 per cent, cycle time fell 32.9 per cent, distance per mission fell 21.6 per cent and run flat events fell 60.9 per cent, while utilisation fell 4.1 per cent and replanning effort rose 18.8 per cent. The conclusions below are stated in the order the evidence was produced, because each rests on an observation reported above rather than on design intent.

1. The gain came from the failure measured first. The diagnosis preceded the proposal, and the largest failure it found was 3.45 run flat events per shift. That is the

quantity that fell to 1.35, and the most visited learned rule is the one producing the fall. The chain is observable at every link: a negative reserve leads to GO_CHARGE, which avoids a tow, which prevents a mission returning to the queue, which is why pending missions halved.

2. A busier fleet is not a better fleet, and this experiment is the evidence. The proposal is less utilised, charges more and delivers considerably more. Utilisation counts motion, not delivery, and the baseline earns part of its utilisation driving toward missions it cannot complete, so in an energy constrained fleet utilisation is a diagnostic metric rather than an objective. Choosing the wrong primary metric here would have led to rejecting the better strategy.

3. Decentralisation survived the change, and the market layer became cheaper. Work stayed balanced with nothing balancing it, the busiest AGV carrying 32.2 per cent of the load against 18.8 for the quietest. Communication fell sharply, 1.18 bids per accepted mission against 7.54 in 20 of 20 seeds, with contested auctions falling from 18.2 to 9.5 per cent, and the four policies diverged on 37.8 per cent of the states they shared. The fleet is not one controller wearing four bodies.

4. The comparison is usable because the two scenarios differ in exactly one layer. Section 4.1 states the difference line by line and the equivalence was verified rather than assumed: identical worlds in 10 of 10 seeds across order streams, pedestrian positions per tick, closed cells per tick and initial batteries, with the same bid utility, weights, auction, routing and battery physics.

5. The result is conditional, and the conditions are known. The advantage holds under scarcity of charging capacity and sustained demand, and it does not survive an arbitrary choice of discount factor or action set. The two development failures, the over penalising discount and the unconditional WAIT, are reported for that reason: they are part of the result, not accidents preceding it.

5.2 Closing the discussion

The answer this project supports is narrower and more useful than a simple yes. The baseline did not fail because it lacked intelligence; it failed because its decision variable was the wrong one. It read the battery at the moment of choosing instead of the battery that would remain once the choice had played out, and Q Learning mattered less as a learning algorithm than as the mechanism that made the right variable, the BFS energy reserve, usable without anyone hand tuning a threshold for it. That is why the improvement is large where the baseline was blind, in run flat events and cycle time, modest where the baseline was already adequate, in load balance, and negative where the proposal never claimed anything, in utilisation and replanning effort. A strategy is intelligent in this setting to the extent that it decides on quantities whose consequences it can actually see, and the measured 60.9 per cent reduction in run flat events is the size of that difference.

5.3 Remaining limitations and future work

Limitations. The state encoding is coarse, since with 288 buckets and battery boundaries at 25, 50 and 75 per cent two different situations can map to the same state. An AGV chooses a class of mission rather than a specific one, and missions are

assigned in publication order, one per AGV per tick, so each round's assignment problem is never solved jointly. Independent learning is nonstationary by construction, which is why convergence stays noisy. Battery is linear in cells driven, no AGV is ever re-tasked mid mission, and the movement arbiter breaks ties by AGV identifier.

Future work, in order of expected value. (1) Bid on a specific mission rather than a class, using Q values over state and mission feature pairs, which requires function approximation and turns the method into a Deep Q Network. (2) Solve the assignment problem of each round jointly with the Hungarian algorithm on the bid matrix. (3) Add a time window reservation table on top of BFS so routes are deconflicted before AGVs meet instead of replanned after they collide, which targets the one metric that clearly worsened. (4) Tune the three bid weights by search, so the negotiation mechanism stops being the only untuned component of an otherwise learned system. (5) Allow mid mission re tasking and charger reservation. (6) Report confidence intervals over several training seeds and extend the evaluation to fleets of six and eight AGVs.

References

- e80 Group (2024). Automated Guided Vehicles and Laser Guided Vehicles for intralogistics. Company technical documentation, provided as the case study for this project.
- Foerster, J., Nardelli, N., Farquhar, G., Afouras, T., Torr, P. H. S., Kohli, P. and Whiteson, S. (2017). Stabilising Experience Replay for Deep Multi Agent Reinforcement Learning. Proceedings of the 34th International Conference on Machine Learning, 1146 to 1155.
- Foramitti, J. (2021). AgentPy: A package for agent based modeling in Python. Journal of Open Source Software, 6(62), 3065.
- Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Hoboken, NJ: Pearson. Chapters 3 and 22.
- Smith, R. G. (1980). The Contract Net Protocol: High Level Communication and Control in a Distributed Problem Solver. IEEE Transactions on Computers, C 29(12), 1104 to 1113.
- Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). Cambridge, MA: MIT Press.
- Sutton, R. S., Precup, D. and Singh, S. (1999). Between MDPs and semi MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence, 112(1 and 2), 181 to 211.
- Tan, M. (1993). Multi Agent Reinforcement Learning: Independent versus Cooperative Agents. Proceedings of the Tenth International Conference on Machine Learning, 330 to 337.
- Tecnologico de Monterrey (2026). TC2008B Modelacion de Sistemas Multiagentes con Graficas Computacionales. Course material, module M3 baseline implementation.
- Watkins, C. J. C. H. and Dayan, P. (1992). Q Learning. Machine Learning, 8(3), 279 to 292.
- Wooldridge, M. (2009). An Introduction to MultiAgent Systems (2nd ed.). Chichester: John Wiley and Sons.

Appendices

Appendix A. Final presentation

Slide deck presented in class, covering the case study, the problem statement, the architecture of the solution, the coordination paradigm, the experimental design, the results with their tradeoffs and the conclusions. Delivered as AGV_MAS_Presentation.pptx and also available in the repository.

Appendix B. Source code in GitHub repository

The complete implementation is contained in one executable notebook, M4_AGV_Intelligent_MAS.ipynb, organised in 25 numbered sections that run top to bottom with no hidden state. Repository access was granted to the course account as required by the brief.

Link: <https://github.com/Briany4717/agv-multi-agent-system.git>

Appendix C. Unity package

The repository includes the Unity package together with the developed scene, containing the warehouse layout, the four Palletrobot AGVs, the Worker pedestrians, the storage racks, the pallet positions, the production line, the truck docks and the charging stations. The AGVUnityBridge.cs component connects as a TCP client to the Python server described in the accompanying README_unity.md, receiving the position, status and battery of every AGV and pedestrian on each simulation tick and instantiating and moving the corresponding models in real time. Installation and execution instructions are provided in README_unity.md inside the repository.

Appendix D. Simulation parameters

Parameter	Value
Plant	48 m by 30 m, 1 m grid, 865 traversable cells
Fleet	4 AGVs, 3 charging stations
Shift length	400 ticks
Orders per shift	about 27, most arriving after tick 0
Dynamic events	3 pedestrians, aisle closures of 15 to 35 ticks
Training	1000 episodes, seeds 10000 to 10999, 56 seconds
Evaluation	20 paired scenarios, seeds 0 to 19, unseen during training
Learning parameters	$\alpha = 0.10$, $\gamma = 0.999$, ϵ from 1.0 to 0.05, greedy at evaluation
Reward weights	$R_{done} = 100$, $c_t = 0.5$, $c_d = 0.3$, $P_{strand} = 150$, $c_{idle} = 4$
Bid weights	$w_d = 2.0$, $w_b = 0.5$, $w_l = 20.0$, identical in both strategies
Battery model	$c_d = 0.6$ per cell, $c_{idle} = 0.15$ per tick, $R_c = 10.0$ per tick, tow 25 ticks

Appendix E. Supplementary material

E.1 Convergence of the training run

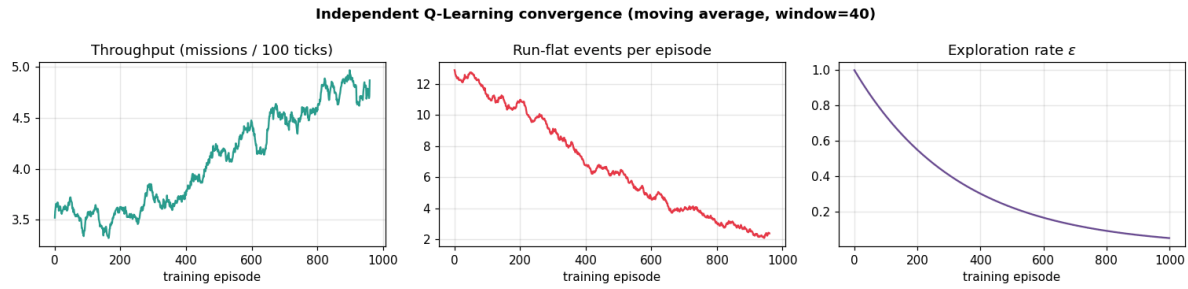


Figure E1. Convergence of independent Q Learning over 1000 training episodes, moving average with a window of 40. Throughput rises from about 2.0 to 4.75 missions per 100 ticks while run flat events fall from 12.6 in the first 50 episodes to 2.7 in the last 200. The residual noise is the signature of independent learning: each agent learns against three moving targets.

E.2 Metrics of the decentralised strategy, graphically

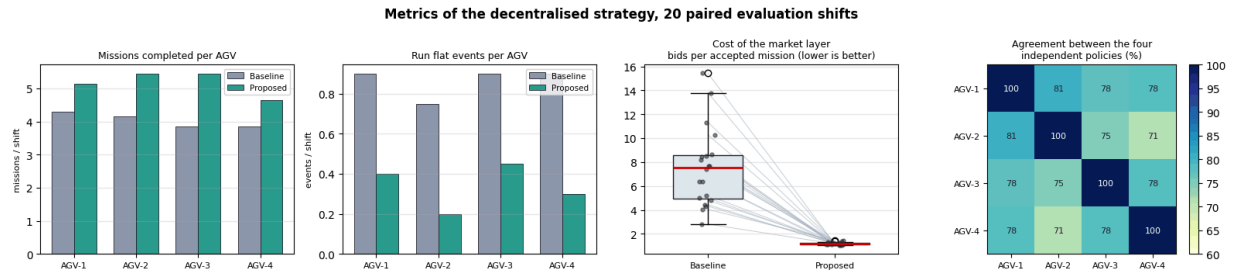


Figure E2. From left to right: missions completed per AGV, run flat events per AGV, cost of the market layer measured as bids per accepted mission with grey lines joining paired seeds, and the pairwise agreement matrix between the four independent learned policies.

E.3 Behaviour learned by the policy

Situation	Learned action	Why it matters
Available, battery 75 per cent or more, work pending, reserve comfortable	BID_NEAREST	Take the job
Available, battery 25 to 50 per cent, work pending, reserve negative	GO_CHARGE	The nearest mission would strand this AGV, so decline it and top up. This single rule drives most of the reduction in run flat events
Parked on a charger, battery 25 to 50 per cent	WAIT	Finish the top up before going back to work
Parked on a charger, battery 75 per cent or more, reserve comfortable	Bid and leave	No longer waits for a full 100 per cent
Available, queue empty	GO_CHARGE	Opportunistic top up instead of standing idle and draining

E.4 Divergence between the four independent policies

Agreement (per cent)	AGV 1	AGV 2	AGV 3	AGV 4
AGV 1	100.0	80.6	77.5	78.0
AGV 2	80.6	100.0	74.8	71.0
AGV 3	77.5	74.8	100.0	78.2
AGV 4	78.0	71.0	78.2	100.0

States visited per agent: 107, 108, 106 and 103 of the 288 possible. States visited by all four agents: 98. States among those where the greedy action differs: 37, that is 37.8 per cent.

E.5 Cost of the centralised alternative

Design	Table entries	Cost of adding a fifth AGV
Four independent Q tables	$4 \times 288 \times 4 = 4608$	one more table, 1152 entries
Single centralised joint policy	$288^4 \times 4^4 = 1\,761\,205\,026\,816$	multiplied by a further factor of 288×4